

## 1. Komponen Proyek (Struktur Kode)

Berikut adalah *breakdown* dari komponen *source code* yang ada di direktori `src/main/java/com/balistic/`:

- **App.java (Main Entry Point / Controller)** Kelas ini berisi *method* `main` yang merupakan titik awal berjalannya program. Bertugas untuk menangani operasi *Input/Output* (I/O) dari pengguna, melakukan *parsing* parameter, dan menghubungkan input tersebut ke komponen logika bisnis.
  - **BallisticCalculator.java (Domain / Business Logic)** Ini adalah kelas krusial yang mengimplementasikan *core logic* aplikasi. Kelas ini berisi sekumpulan algoritma dan rumus fisika (seperti perhitungan lintasan parabola, titik tertinggi, dan jarak terjauh). Kelas ini didesain independen dari I/O agar mudah di-*mock* dan di-*test* (*testable*).
  - **Projectile.java (Data Model / POJO)** Kelas ini bertindak sebagai *Data Transfer Object* (DTO) atau sekadar *Plain Old Java Object* (POJO). Fungsinya merepresentasikan *state* dari proyektil atau objek yang ditembakkan (misalnya menyimpan atribut massa, koordinat saat ini, dll), sehingga perpindahan data antar *method* lebih terstruktur.
  - **UnitConverter.java (Utility / Helper Class)** Kelas *helper* yang berisikan sekumpulan *static method* murni. Berfungsi untuk melakukan normalisasi tipe data dan metrik, misalnya mengubah besaran metrik (seperti *km/h* menjadi *m/s*) sebelum data tersebut diproses oleh `BallisticCalculator`.
- 

## 2. Penjelasan 4 Workflow CI/CD (GitHub Actions)

Di dalam direktori `.github/workflows/ci-cd.yml`, terdapat otomasi *pipeline* yang terbagi menjadi 4 komponen utama. Keempat komponen ini merepresentasikan siklus hidup CI/CD standar di industri:

### 1. Continuous Integration (Build & Compile)

- **Fungsi:** Tahap paling awal untuk memastikan *source code* tidak mengalami *syntax error* dan semua *dependency* (yang didefinisikan di `pom.xml`) dapat diunduh dengan benar di *environment* yang bersih (*clean environment* seperti kontainer Ubuntu).
- **Eksekusi:** Menjalankan perintah `mvn compile`. Jika tahap ini gagal, berarti kode tersebut bahkan tidak bisa di-*compile*.

### 2. Continuous Testing (Automated Test & Coverage)

- **Fungsi:** Memastikan fitur baru tidak merusak fitur yang sudah ada (*preventing regression bugs*). Tahap ini menjalankan semua *Unit Test* menggunakan **JUnit**. Selain itu, tahap ini menjalankan **JaCoCo** untuk menghitung *Code Coverage* (memastikan bahwa minimal 70% baris kode telah ter-eksekusi oleh skenario pengujian).
- **Eksekusi:** Menjalankan `mvn test jacoco:report` dan `mvn jacoco:check`.

### 3. Continuous Inspection (Static Code Analysis)

- **Fungsi:** Melakukan *Static Application Security Testing* (SAST) dan menjaga kualitas kode (*Code Quality*). Tahap ini mengecek apakah gaya penulisan kode sudah rapi dan

sesuai konvensi menggunakan **Checkstyle**, serta mendeteksi potensi *code smells* atau variabel yang tidak efisien menggunakan **PMD**.

- **Eksekusi:** Menjalankan `mvn checkstyle:check` dan `mvn pmd:check`.

#### 4. **Continuous Deployment / Delivery (Packaging & Release)**

- **Fungsi:** Jika kode berhasil melewati ketiga tahap verifikasi di atas dengan status *Pass*, *pipeline* akan beralih ke tahap perilisan. Maven (*maven-assembly-plugin*) akan mem-paketkan kode beserta seluruh *dependency*-nya menjadi satu *Executable Fat JAR*.
- **Eksekusi:** Membuat versi rilis secara otomatis di GitHub Releases (*Automated Release*), melampirkan *artifact* .jar tersebut agar siap di-*deploy* atau diunduh oleh pengguna (*End-User*).